

## Logical Query Processing Phases

This section covers logical query processing and the phases involved. Don't worry if some of the concepts discussed here aren't clear yet. Subsequent chapters in this Training Kit provide more detail, and after you go over those, this topic should make more sense. To make sure you really understand these concepts, make a first pass over the topic now and then revisit it later after going over Chapters 2 through 5.

The main statement used to retrieve data in T-SQL is the SELECT statement. Following are the main query clauses specified in the order that you are supposed to type them (known as "keyed-in order"):

1. SELECT
2. FROM
3. WHERE
4. GROUP BY
5. HAVING
6. ORDER BY

But as mentioned, the logical query processing order, which is the conceptual interpretation order, is different. It starts with the FROM clause. Here is the logical query processing order of the six main query clauses:

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

Each phase operates on one or more tables as inputs and returns a virtual table as output.

The output table of one phase is considered the input to the next phase. This is in accord with operations on relations that yield a relation. Note that if an ORDER BY is specified, the result isn't relational. This fact has implications that are discussed later in this Training Kit, in Chapter 3, "Filtering and Sorting Data," and Chapter 4, "Combining Sets."

Consider the following query as an example.

```
SELECT country, YEAR(hiredate) AS yearhired, COUNT(*) AS numemployees
FROM HR.Employees
WHERE hiredate >= '20030101'
GROUP BY country, YEAR(hiredate)
HAVING COUNT(*) > 1
ORDER BY country , yearhired DESC;
```

This query is issued against the HR.Employees table. It filters only employees that were hired in or after the year 2003. It groups the remaining employees by country and the hire year. It keeps only groups with more than one employee. For each qualifying group, the query returns the hire year and count of employees, sorted by country and hire year, in descending order.

The following sections provide a brief description of what happens in each phase according to logical query processing.

### 1. Evaluate the FROM Clause

In the first phase, the FROM clause is evaluated. That's where you indicate the tables you want to query and table operators like joins if applicable. If you need to query just one table, you indicate the table name as the input table in this clause. Then, the output of this phase is a table result with all rows from the input table. That's the case in the following query: the input is the HR.Employees table (nine rows), and the output is a table result with all nine rows (only a subset of the attributes are shown).

```
empid hiredate country
-----
1 2002-05-01 USA
2 2002-08-14 USA
3 2002-04-01 USA
4 2003-05-03 USA
5 2003-10-17 UK
6 2003-10-17 UK
7 2004-01-02 UK
8 2004-03-05 USA
9 2004-11-15 UK
```

### 2. Filter Rows Based on the WHERE Clause

The second phase filters rows based on the predicate in the WHERE clause. Only rows for which the predicate evaluates to true are returned.

In this query, the WHERE filtering phase filters only rows for employees hired on or after

January 1, 2003. Six rows are returned from this phase and are provided as input to the next one. Here's the result of this phase.

```
empid hiredate country
-----
4 2003-05-03 USA
5 2003-10-17 UK
6 2003-10-17 UK
7 2004-01-02 UK
8 2004-03-05 USA
9 2004-11-15 UK
```

A typical mistake made by people who don't understand logical query processing is attempting to refer in the WHERE clause to a column alias defined in the SELECT clause. This isn't allowed because the WHERE clause is evaluated before the SELECT clause. As an example, consider the following query.

```
SELECT country, YEAR(hiredate) AS yearhired
FROM HR.Employees
WHERE yearhired >= 2003;
```

This query fails with the following error.

```
Msg 207, Level 16, State 1, Line 3
Invalid column name 'yearhired'.
```

If you understand that the WHERE clause is evaluated before the SELECT clause, you realize that this attempt is wrong because at this phase, the attribute yearhired doesn't yet exist. You can indicate the expression YEAR(hiredate) >= 2003 in the WHERE clause. Better yet, for optimization reasons that are discussed in Chapter 3 and Chapter 15, "Implementing Indexes and Statistics," use the form hiredate >= '20030101' as done in the original query.

### 3. Group Rows Based on the GROUP BY Clause

This phase defines a group for each distinct combination of values in the grouped elements from the input table. It then associates each input row to its respective group. The query you've been working with groups the rows by country and YEAR(hiredate). Within the six rows in the input table, this step identifies four groups. Here are the groups and the detail rows that are associated with them (redundant information removed for purposes of illustration).

| group<br>country | group<br>YEAR(hiredate) | detail<br>empid | detail<br>country | detail<br>hiredate |
|------------------|-------------------------|-----------------|-------------------|--------------------|
| UK               | 2003                    | 5               | UK                | 2003-10-17         |
|                  |                         | 6               | UK                | 2003-10-17         |
| UK               | 2004                    | 7               | UK                | 2004-01-02         |
|                  |                         | 9               | UK                | 2004-11-15         |
| USA              | 2003                    | 4               | USA               | 2003-05-03         |
| USA              | 2004                    | 8               | USA               | 2004-03-05         |

As you can see, the group UK, 2003 has two associated detail rows with employees 5 and 6; the group for UK, 2004 also has two associated detail rows with employees 7 and 9; the group for USA, 2003 has one associated detail row with employee 4; the group for USA, 2004 also has one associated detail row with employee 8.

The final result of this query has one row representing each group (unless filtered out). Therefore, expressions in all phases that take place after the current grouping phase are somewhat limited. All expressions processed in subsequent phases must guarantee a single value per group. If you refer to an element from the GROUP BY list (for example, country), you already have such a guarantee, so such a reference is allowed. However, if you want to refer to an element that is not part of your GROUP BY list (for example, empid), it must be contained within an aggregate function like MAX or SUM. That's because multiple values are possible in the element within a single group, and the

only way to guarantee that just one will be returned is to aggregate the values. For more details on grouped queries, see Chapter 5, “Grouping and Windowing.”

#### 4. Filter Rows Based on the HAVING Clause

This phase is also responsible for filtering data based on a predicate, but it is evaluated after the data has been grouped; hence, it is evaluated per group and filters groups as a whole. As is usual in T-SQL, the filtering predicate can evaluate to true, false, or unknown. Only groups for which the predicate evaluates to true are returned from this phase. In this case, the HAVING clause uses the predicate `COUNT(*) > 1`, meaning filter only country and hire year groups that have more than one employee. If you look at the number of rows that were associated with each group in the previous step, you will notice that only the groups UK, 2003 and UK, 2004 qualify. Hence, the result of this phase has the following remaining groups, shown here with their associated detail rows.

| group<br>country | group<br>YEAR(hiredate) | detail<br>empid | detail<br>country | detail<br>hiredate |
|------------------|-------------------------|-----------------|-------------------|--------------------|
| UK               | 2003                    | 5               | UK                | 2003-10-17         |
|                  |                         | 6               | UK                | 2003-10-17         |
| UK               | 2004                    | 7               | UK                | 2004-01-02         |
|                  |                         | 9               | UK                | 2004-11-15         |

**Difference between WHERE and HAVING clauses:** The WHERE clause is evaluated before rows are grouped, and therefore is evaluated per row. The HAVING clause is evaluated after rows are grouped, and therefore is evaluated per group.

#### 5. Process the SELECT Clause

The fifth phase is the one responsible for processing the SELECT clause. What’s interesting about it is the point in logical query processing where it gets evaluated—almost last. That’s interesting considering the fact that the SELECT clause appears first in the query.

This phase includes two main steps.

The first step is evaluating the expressions in the SELECT list and producing the result attributes. This includes assigning attributes with names if they are derived from expressions. Remember that if a query is a grouped query, each group is represented by a single row in the result. In the query, two groups remain after the processing of the HAVING filter. Therefore, this step generates two rows. In this case, the SELECT list returns for each country and hire year group a row with the following attributes: country, YEAR(hiredate) aliased as yearhired, and COUNT(\*) aliased as numemployees.

The second step in this phase is applicable if you indicate the DISTINCT clause, in which case this step removes duplicates. Remember that T-SQL is based on multiset theory more than it is on set theory, and therefore, if duplicates are possible in the result, it’s your responsibility to remove those with the DISTINCT clause. In this query’s case, this step is inapplicable.

Here’s the result of this phase in the query.

| country | yearhired | numemployees |
|---------|-----------|--------------|
| UK      | 2003      | 2            |
| UK      | 2004      | 2            |

If you need a reminder of what the query looks like, here it is again.

```
SELECT country, YEAR(hiredate) AS yearhired, COUNT(*) AS numemployees
FROM HR.Employees
WHERE hiredate >= '20030101'
GROUP BY country, YEAR(hiredate)
HAVING COUNT(*) > 1
ORDER BY country , yearhired DESC;
```

The fifth phase returns a relational result. Therefore, the order of the rows isn’t guaranteed. In this query’s case, there is an ORDER BY clause that guarantees the order in the result, but this will be discussed when the next phase is

described. What's important to note is that the outcome of the phase that processes the SELECT clause is still relational.

Also, remember that this phase assigns column aliases, like `yearhired` and `numemployees`. This means that newly created column aliases are not visible to clauses processed in previous phases, like FROM, WHERE, GROUP BY, and HAVING.

Note that an alias created by the SELECT phase isn't even visible to other expressions that appear in the same SELECT list. For example, the following query isn't valid.

```
SELECT empid, country, YEAR(hiredate) AS yearhired, yearhired - 1 AS prevyear
FROM HR.Employees;
```

This query generates the following error.

```
Msg 207, Level 16, State 1, Line 1
Invalid column name 'yearhired'.
```

The reason that this isn't allowed is that, conceptually, T-SQL evaluates all expressions that appear in the same logical query processing phase in an all-at-once manner. Note the use of the word *conceptually*. SQL Server won't necessarily physically process all expressions at the same point in time, but it has to produce a result as if it did. This behavior is different than many other programming languages where expressions usually get evaluated in a left-to-right order, making a result produced in one expression visible to the one that appears to its right. But T-SQL is different.

## 6. Handle Presentation Ordering

The sixth phase is applicable if the query has an ORDER BY clause. This phase is responsible for returning the result in a specific presentation order according to the expressions that appear in the ORDER BY list. The query indicates that the result rows should be ordered first by country (in ascending order by default), and then by numemployees, descending, yielding the following output.

| country | yearhired | numemployees |
|---------|-----------|--------------|
| UK      | 2004      | 2            |
| UK      | 2003      | 2            |

Notice that the ORDER BY clause is the first and only clause that is allowed to refer to column aliases defined in the SELECT clause. That's because the ORDER BY clause is the only one to be evaluated after the SELECT clause.

Unlike in previous phases where the result was relational, the output of this phase isn't relational because it has a guaranteed order. The result of this phase is what standard SQL calls a cursor. Note that the use of the term cursor here is conceptual. T-SQL also supports an object called a cursor that is defined based on a result of a query, and that allows fetching rows one at a time in a specified order.

You might care about returning the result of a query in a specific order for presentation purposes or if the caller needs to consume the result in that manner through some cursor mechanism that fetches the rows one at a time. But remember that such processing isn't relational. If you need to process the query result in a relational manner—for example, define a table expression like a view based on the query (details later in Chapter 4)—the result will need to be relational. Also, sorting data can add cost to the query processing. If you don't care about the order in which the result rows are returned, you can avoid this unnecessary cost by not adding an ORDER BY clause.

A query may specify the TOP or OFFSET-FETCH filtering options. If it does, the same ORDER BY clause that is normally used to define presentation ordering also defines which rows to filter for these options. It's important to note that such a filter is processed after the SELECT phase evaluates all expressions and removes duplicates (in case a DISTINCT clause was specified). You might even consider the TOP and OFFSET-FETCH filters as being processed in their own phase number 7. The query doesn't indicate such a filter, and therefore, this phase is inapplicable in this case.